

OS/161 Assignment 1 Report

Synchronization Problems

Name: _____
Student ID: _____
Course: Operating Systems Lab

July 1, 2026

Abstract

This report describes the implementation of Assignment 1 for OS/161. The assignment focuses on synchronization among kernel threads. The implemented parts are concurrent mathematics, bounded-buffer producer/consumer, and bar synchronization. The local OS/161 lock and condition-variable primitives were also completed because the assignment solutions depend on working synchronization primitives.

Contents

1	Assignment Overview	3
2	Build and Run Procedure	3
3	Synchronization Primitives	3
3.1	Lock Implementation	4
3.2	Condition Variable Implementation	4
4	Part 1: Concurrent Mathematics	4
4.1	Problem	4
4.2	Shared Data	4
4.3	Solution	5
4.4	Why It Works	5
5	Part 2: Bounded-Buffer Producer/Consumer	5
5.1	Problem	5
5.2	Shared Data	5
5.3	Solution	5
5.4	Why It Works	6
6	Part 3: Bar Synchronization	6
6.1	Problem	6
6.2	Shared Data	6
6.3	Solution	6

6.4 Why It Works	7
7 Test Results	7
8 Conclusion	10
A Source Code: math.c	11
B Source Code: producerconsumer.c	17
C Source Code: bar.c	19
D Source Code: bar_driver.c	23
E Source Code: synch.c	26

1 Assignment Overview

The goal of this assignment is to identify shared resources accessed by multiple OS/161 kernel threads and protect them using synchronization primitives. Without synchronization, interleavings caused by the scheduler can produce lost updates, inconsistent statistics, overwritten buffer entries, empty reads, or deadlock.

The implemented test commands are shown in Table 1.

Command	Problem	Main file
1a	Concurrent mathematics	kern/asst1/math.c
1b	Producer/consumer buffer	kern/asst1/producerconsumer.c
1c	Bar synchronization	kern/asst1/bar.c

Table 1: Assignment test commands

2 Build and Run Procedure

The kernel was configured and built for ASST1. In the Docker-based environment, the build script can be used directly:

```
cd /root/cs350-os161
bash build-and-run-kernel.sh
```

The important manual build commands are:

```
cd /root/cs350-os161/os161-1.99
./configure --ostree=/root/cs350-os161/root --toolprefix=cs350-
cd kern/conf
./config ASST1
cd ../compile/ASST1
bmake depend
bmake
bmake install
```

The System/161 configuration was updated to use 2 MiB of RAM:

```
31 mainboard    ramsize=2097152    cpus=1
```

The tests were run from the OS/161 kernel menu:

```
1a
1b
1c
q
```

3 Synchronization Primitives

The local OS/161 tree still had incomplete implementations of locks and condition variables. These primitives were completed in kern/thread/synch.c and declared in kern/include/synch

3.1 Lock Implementation

Each lock stores:

- a wait channel for blocked threads,
- a spinlock protecting the lock's internal state,
- a boolean flag showing whether the lock is held,
- a pointer to the thread currently holding the lock.

When a thread calls `lock_acquire`, it first checks that it is not already holding the lock. It then takes the internal spinlock. If the lock is already held, the thread sleeps on the wait channel. When the lock becomes free, the thread marks it as held and records itself as the owner.

When `lock_release` is called, the implementation checks that the current thread owns the lock, clears the owner, marks the lock free, and wakes one sleeping thread.

3.2 Condition Variable Implementation

Condition variables use wait channels. The operation `cv_wait` requires the caller to hold the associated lock. It locks the CV wait channel, releases the caller's lock, sleeps, and then reacquires the lock after waking. This follows Mesa-style condition-variable semantics, where a woken thread must recheck the condition after it runs.

4 Part 1: Concurrent Mathematics

4.1 Problem

The math test creates ten adder threads. All threads increment the same global counter until the total reaches 10000. Without synchronization, two threads can read the same old value, both compute the same next value, and one increment is lost. The printed statistics can also become inconsistent with the final counter.

4.2 Shared Data

The shared data in this part is:

- `counter`: the global count incremented by all adders,
- `finished`: semaphore used by the main thread to wait,
- `math_lock`: lock protecting the counter critical section.

The array `adder_counters[]` is not shared in a harmful way because each thread updates only its own array index.

4.3 Solution

The critical section begins before reading `counter` and ends after the increment and consistency check value are recorded. This makes the operation “read, check, increment, read back” atomic with respect to other adder threads.

The call to `math_test_2` is intentionally outside the lock, following the assignment comment that it should not execute mutually exclusively. This keeps the critical section small.

4.4 Why It Works

Only one adder can inspect and increment the global counter at a time. Therefore no two threads can consume the same counter value, and the final total cannot skip or duplicate increments. The semaphore `finished` is signalled once by each worker, so the main thread waits until all ten adders exit before printing statistics.

5 Part 2: Bounded-Buffer Producer/Consumer

5.1 Problem

Producer threads generate data items and consumer threads remove them. The buffer has a fixed size. A producer must block when the buffer is full, and a consumer must block when the buffer is empty. Busy waiting is not allowed.

5.2 Shared Data

The shared data is:

- `item_buffer[BUFFER_SIZE]`: circular FIFO buffer,
- `producer_index`: next insert position,
- `consumer_index`: next remove position.

The synchronization objects are:

- `empty`: counts free buffer slots,
- `full`: counts occupied buffer slots,
- `mutex`: binary semaphore protecting index and buffer updates.

5.3 Solution

The producer performs:

```
P(empty);
P(mutex);
insert item at producer_index;
advance producer_index circularly;
V(mutex);
V(full);
```

The consumer performs:

```
P(full);
P(mutex);
remove item at consumer_index;
advance consumer_index circularly;
V(mutex);
V(empty);
```

5.4 Why It Works

The **empty** semaphore prevents insertion into a full buffer. The **full** semaphore prevents removal from an empty buffer. The **mutex** semaphore ensures that only one thread at a time changes the buffer indices or buffer entries. Because both indices wrap using modulo **BUFFER_SIZE**, the buffer behaves as a circular FIFO queue.

6 Part 3: Bar Synchronization

6.1 Problem

The bar model contains customer threads and bartender threads. Customers place drink orders and wait until the correct glass is returned. Bartenders take orders, mix drinks using shared bottles, and eventually go home after all customers are finished. The solution must prevent lost orders, wrong drinks, bottle races, deadlock, and bartenders waiting forever after the bar closes.

6.2 Shared Data

The shared data in the bar solution is:

- the bounded order queue,
- queue indices **order_in** and **order_out**,
- the number of customers still active,
- the drink bottles,
- bottle usage statistics.

6.3 Solution

Customers produce **struct bar_order** objects and bartenders consume them from a bounded circular queue. The queue uses the same pattern as the producer/consumer problem: **empty_orders** counts free order slots, **full_orders** counts queued orders, and **order_lock** protects the queue indices.

Each order contains a private completion semaphore named **finished**. After a customer queues an order, it sleeps on this semaphore. A bartender signals it after copying the requested ingredients into the glass contents.

Each bottle has its own lock. When a drink needs one or more bottles, the bottle numbers are sorted and the locks are acquired in increasing order. This allows parallel mixing when different bottles are used while avoiding circular wait when a drink needs multiple bottles.

When the last customer leaves, the solution enqueues one invalid order for each bartender. These invalid orders are closing messages. A bartender that receives one stops waiting and goes home.

6.4 Why It Works

The order queue prevents lost or overwritten orders. A customer cannot leave before receiving a drink because it blocks on its order semaphore. Bottle locks protect shared bottle use, and the sorted acquisition order prevents deadlock. The invalid closing orders ensure every bartender exits after all customers are finished.

7 Test Results

The kernel was run in System/161 and the three assignment commands completed successfully.

```
OS/161 kernel [? for menu]: 1a
Starting 10 adder threads
Adder threads performed 10000 adds
The adders performed 10000 increments overall (expected 10000)

OS/161 kernel [? for menu]: 1b
run_producerconsumer: starting up
All producer threads have exited.
Consumer finished normally
Consumer finished normally
Consumer finished normally
Consumer finished normally
Consumer finished normally

OS/161 kernel [? for menu]: 1c
S 2 going home after mixing 42 drinks
S 1 going home after mixing 28 drinks
S 0 going home after mixing 30 drinks
Bottle 1 used for 100 doses
Bottle 2 used for 0 doses
Bottle 3 used for 0 doses
Bottle 4 used for 0 doses
Bottle 5 used for 0 doses
Bottle 6 used for 0 doses
Bottle 7 used for 0 doses
Bottle 8 used for 0 doses
Bottle 9 used for 0 doses
Bottle 10 used for 0 doses
The bar is closed, bye!!!
```

```

OS/161 kernel [? for menu]: ?
OS/161 kernel menu
  [?o] Operations menu          [sp1] Whale Mating
  [?t] Tests menu              [sp2] Cat/mouse
  [1a] Simple math synchronisation [sp3] Traffic
  [1b] Producer/consumer problem [kh] Kernel heap stats
  [1c] Two locks               [q] Quit and shut down

Operation took 0.119271280 seconds
OS/161 kernel [? for menu]: 
[os161] 0:sys161*

```

Figure 1: System/161 started with GDB attached

```

OS/161 kernel [? for menu]: 1a
Starting 10 adder threads
Adder threads performed 10000 adds
Adder 0 performed 1013 increments.
Adder 1 performed 961 increments.
Adder 2 performed 991 increments.
Adder 3 performed 1010 increments.
Adder 4 performed 1089 increments.
Adder 5 performed 992 increments.
Adder 6 performed 1044 increments.
Adder 7 performed 982 increments.
Adder 8 performed 993 increments.
Adder 9 performed 925 increments.
The adders performed 10000 increments overall (expected 10000)
Operation took 3.759909760 seconds
OS/161 kernel [? for menu]: 
[os161] 0:sys161*

```

Figure 2: Successful output for 1a Concurrent Mathematics


```
OS/161 kernel [? for menu]: 1b
run_producerconsumer: starting up
Waiting for producer threads to exit...
Producer started
Consumer started
Consumer started
Consumer started
Consumer started
Consumer started
Consumer started
Producer started
Producer finished
Producer finished
All producer threads have exited.
Consumer finished normally
Consumer finished normally
Consumer finished normally
Consumer finished normally
Consumer finished normally
Operation took 0.286187880 seconds
OS/161 kernel [? for menu]:
```

Figure 3: Successful output for 1b Producer/Consumer

```
Operation took 0.117350560 seconds
OS/161 kernel [? for menu]: 1c
S 0 going home after mixing 45 drinks
S 2 going home after mixing 38 drinks
S 1 going home after mixing 17 drinks
Bottle 1 used for 100 doses
Bottle 2 used for 0 doses
Bottle 3 used for 0 doses
Bottle 4 used for 0 doses
Bottle 5 used for 0 doses
Bottle 6 used for 0 doses
Bottle 7 used for 0 doses
Bottle 8 used for 0 doses
Bottle 9 used for 0 doses
Bottle 10 used for 0 doses
The bar is closed, bye!!!
Operation took 0.487920640 seconds
OS/161 kernel [? for menu]: 
[os161] 0:sys161*
```

Figure 4: Successful output for 1c Bar Synchronization

8 Conclusion

The assignment demonstrates the core synchronization problems that arise in a kernel with concurrent threads. The math part requires mutual exclusion around a shared counter. The producer/consumer part requires both mutual exclusion and counting synchronization to coordinate buffer capacity. The two-locks part requires a consistent lock acquisition order to prevent deadlock. The completed OS/161 lock and condition-variable primitives provide the foundation required for these solutions and for later synchronization work.

A Source Code: math.c

```
1  #include "opt-synchprobs.h"
2  #include "math_tester.h"
3  #include <types.h>
4  #include <lib.h>
5  #include <test.h>
6  #include <thread.h>
7  #include <synch.h>
8
9
10
11 enum {
12     NADDERS = 10,          /* the number of adder threads */
13     NADDS   = 10000, /* the number of overall increments to
14                        perform */
15 };
16
17 /*
18  * Declare the counter variable that all the adder() threads
19  * increment
20  *
21  * Declaring it "volatile" instructs the compiler to always (re)
22  * read the
23  * variable from memory and not optimise by removing memory
24  * references
25  * and re-using the content of a register.
26  */
27 volatile unsigned long int counter;
28
29 /*
30  * Declare an array of adder counters to count per-thread
31  * increments. These are used for printing statistics.
32  */
33 unsigned long int adder_counters[NADDERS];
34
35 /*
36  * We use a semaphore to wait for adder() threads to finish
37  */
38 struct semaphore *finished;
39
40 /*
41  *
42  * *****
43  *
44  * ADD YOUR OWN VARIABLES HERE AS NEEDED
45  *
46  * *****
47  */
```

```

42  */
43
44  struct lock *math_lock;
45
46
47
48  /*
49  * adder()
50  *
51  * Each adder thread simply keeps incrementing the counter until
52  * we
53  * hit the max value.
54  *
55  * *****
56  * YOU NEED TO INSERT SYNCHRONISATION PRIMITIVES APPROPRIATELY
57  * TO ENSURE COUNTING IS CORRECTLY PERFORMED.
58  * *****
59  *
60  * You should not re-write the existing code.
61  *
62  * + Only the correct number of increments are performed
63  * + Ensure x+1 == x+1
64  * + Ensure that the statistics kept match the number of
65  *   increments
66  *   performed.
67  * + only synchronise the critical section, no more.
68  *
69  */
70  static void adder(void * unusedpointer, unsigned long addernumber
71  )
72  {
73      unsigned long int a, b;
74      int flag = 1;
75
76      /*
77      * Avoid unused variable warnings.
78      */
79      (void) unusedpointer; /* remove this line if variable is
80      used */
81
82      while (flag) {
83          /* loop doing increments until we achieve the
84          overall number
85          of increments */

```

```

84
85     lock_acquire(math_lock);
86
87     a = counter;
88     if (a < NADDS) {
89
90         counter = counter + 1;
91
92         math_test_1(addernumber); /* We use this
93                                     for testing, please leave this here.
94                                     */
95
96         b = counter;
97
98         lock_release(math_lock);
99
100        math_test_2(addernumber); /* We use this
101                                    for testing, please leave this here.
102                                    * Also note it
103                                    should NOT
104                                    execute
105                                    mutually
106                                    exclusively
107                                    */
108
109        /*
110         * now count the number of increments we
111         * perform for statistics.
112         * addernumber is effectively the thread
113         * number of this thread.
114         * Note: An individual array location is
115         * only accessed by a single
116         * thread and has no concurrency issues.
117         */
118        adder_counters[addernumber]++;
119
120        /* check we are getting sane results.
121         * This should not print in a correct
122         * solution.
123         */
124        if (a + 1 != b) {
125            kprintf("In thread %ld, %ld + 1
126                    == %ld?\n",
127                    addernumber, a, b) ;
128        }
129    } else {
130        lock_release(math_lock);
131        flag = 0;
132    }
133 }

```

```

123         /* signal the main thread we have finished and then exit
124             */
125         V(finished);
126
127         thread_exit();
128     }
129
130     /*
131     * maths()
132     *
133     * This function:
134     * + Initialises the counter variables
135     * + Creates a semaphore to wait for adder threads to complete
136     * + Starts the define number of adder threads
137     * + waits, prints statistics, cleans up, and exits
138     */
139
140     int maths (int data1, char **data2)
141     {
142         int index, error;
143         unsigned long int sum;
144
145         /*
146         * Avoid unused variable warnings from the compiler.
147         */
148         (void) data1;
149         (void) data2;
150
151         /* initialise the counter before the threads start */
152
153         counter = 0;
154         for (index = 0; index < NADDERS; index++) {
155             adder_counters[index] = 0;
156         }
157
158         /* create a semaphore to allow main thread to wait on
159            workers */
160
161         finished = sem_create("finished", 0);
162
163         if (finished == NULL) {
164             panic("maths: sem create failed");
165         }
166
167         /*
168         *
169         *****
170
171         * INSERT ANY INITIALISATION CODE YOU REQUIRE HERE

```

169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214

```

    *
    *****

*/

math_lock = lock_create("math lock");

if (math_lock == NULL) {
    panic("maths: lock create failed");
}

/*
 * Start NADDERS adder() threads.
 */

kprintf("Starting %d adder threads\n", NADDERS);

for (index = 0; index < NADDERS; index++) {

    error = thread_fork("adder thread", NULL, &adder,
        NULL, index);

    /*
     * panic() on error as we can't progress if we
     * can't create threads.
     */

    if (error) {
        panic("adder: thread_fork failed: %s\n",
            strerror(error));
    }
}

/*
 * Wait until the adder threads complete by waiting on
 * the semaphore NADDER times.
 */

for (index = 0; index < NADDERS; index++) {
    P(finished);
}

kprintf("Adder threads performed %ld adds\n", counter);

/* Print out some statistics, they should add up */
sum = 0;
for (index = 0; index < NADDERS; index++) {
    sum += adder_counters[index];
}
```

```

215         kprintf("Adder %d performed %ld increments.\n",
216                 index,
217                 adder_counters[index]);
218     }
219     kprintf("The adders performed %ld increments overall (
220             expected %d)\n", sum, NADDS);
221
222     /*
223     *
224     * *****
225
226     * INSERT ANY CLEANUP CODE YOU REQUIRE HERE
227     *
228     * *****
229
230     */
231
232     lock_destroy(math_lock);
233
234     /* clean up the semaphore we allocated earlier */
235     sem_destroy(finished);
236     return 0;
237 }

```


B Source Code: producerconsumer.c

```
1  /* This file will contain your solution. Modify it as you wish.
   */
2  #include <types.h>
3  #include <lib.h>
4  #include <synch.h>
5  #include "producerconsumer_driver.h"
6
7  /* Declare any variables you need here to keep track of and
8     synchronise your bounded. A sample declaration of a buffer is
9     shown
10     below. It is an array of pointers to items.
11
12     You can change this if you choose another implementation.
13     However, you should not have a buffer bigger than BUFFER_SIZE
14 */
15 static data_item_t *item_buffer[BUFFER_SIZE];
16 static int producer_index, consumer_index;
17
18 static struct semaphore *mutex, *empty, *full;
19
20
21 /* consumer_receive() is called by a consumer to request more
22    data. It
23    should block on a sync primitive if no data is available in
24    your
25    buffer. It should not busy wait! */
26
27 data_item_t * consumer_receive(void)
28 {
29     data_item_t * item;
30
31     P(full);
32     P(mutex);
33
34     /* Remove the oldest item from the circular buffer. */
35     item = item_buffer[consumer_index];
36     consumer_index = (consumer_index + 1) % BUFFER_SIZE;
37
38     V(mutex);
39     V(empty);
40
41     return item;
42 }
43
44 /* producer_send() is called by a producer to store data in your
45    bounded buffer. It should block on a sync primitive if no
46    space is
```

```

45     available in your buffer. It should not busy wait!*/
46
47 void producer_send(data_item_t *item)
48 {
49     P(empty);
50     P(mutex);
51
52     /* Add the newest item to the circular buffer. */
53     item_buffer[producer_index] = item;
54     producer_index = (producer_index + 1) % BUFFER_SIZE;
55     V(mutex);
56     V(full);
57 }
58
59
60
61
62 /* Perform any initialisation (e.g. of global data) you need
63    here. Note: You can panic if any allocation fails during setup
64    */
65 void producerconsumer_startup(void)
66 {
67
68     producer_index = 0;
69     consumer_index = 0;
70
71     mutex = sem_create("mutex", 1);
72     KASSERT(mutex != 0);
73
74     empty = sem_create("empty", BUFFER_SIZE);
75     KASSERT(empty != 0);
76
77     full = sem_create("full", 0);
78     KASSERT(full != 0);
79 }
80
81 /* Perform any clean-up you need here */
82 void producerconsumer_shutdown(void)
83 {
84     sem_destroy(mutex);
85     sem_destroy(empty);
86     sem_destroy(full);
87 }

```

C Source Code: bar.c

```
1  /* Bar synchronization solution. */
2  #include <types.h>
3  #include <lib.h>
4  #include <synch.h>
5  #include "bar_driver.h"
6
7  static struct bar_order *order_buffer[ORDER_BUFFER_SIZE];
8  static int order_in;
9  static int order_out;
10 static int customers_left;
11
12 static struct semaphore *empty_orders;
13 static struct semaphore *full_orders;
14 static struct lock *order_lock;
15 static struct lock *bottle_locks[NBOTTLES + 1];
16
17 static void
18 enqueue_order(struct bar_order *order)
19 {
20     P(empty_orders);
21     lock_acquire(order_lock);
22     order_buffer[order_in] = order;
23     order_in = (order_in + 1) % ORDER_BUFFER_SIZE;
24     lock_release(order_lock);
25     V(full_orders);
26 }
27
28 static struct bar_order *
29 dequeue_order(void)
30 {
31     struct bar_order *order;
32
33     P(full_orders);
34     lock_acquire(order_lock);
35     order = order_buffer[order_out];
36     order_out = (order_out + 1) % ORDER_BUFFER_SIZE;
37     lock_release(order_lock);
38     V(empty_orders);
39
40     return order;
41 }
42
43 void
44 bar_open(void)
45 {
46     int i;
47
48     order_in = 0;
49     order_out = 0;
```

```

50     customers_left = BAR_CUSTOMERS;
51
52     empty_orders = sem_create("bar empty orders",
53                               ORDER_BUFFER_SIZE);
54     KASSERT(empty_orders != NULL);
55
56     full_orders = sem_create("bar full orders", 0);
57     KASSERT(full_orders != NULL);
58
59     order_lock = lock_create("bar order lock");
60     KASSERT(order_lock != NULL);
61
62     for (i = 1; i <= NBOTTLES; i++) {
63         bottle_locks[i] = lock_create("bar bottle lock");
64         KASSERT(bottle_locks[i] != NULL);
65     }
66
67 void
68 bar_close(void)
69 {
70     int i;
71
72     for (i = 1; i <= NBOTTLES; i++) {
73         lock_destroy(bottle_locks[i]);
74     }
75
76     lock_destroy(order_lock);
77     sem_destroy(full_orders);
78     sem_destroy(empty_orders);
79
80     kprintf("The bar is closed, bye!!!\n");
81 }
82
83 void
84 bar_customer_order(struct bar_order *order)
85 {
86     enqueue_order(order);
87     P(order->finished);
88 }
89
90 void
91 bar_customer_done(void)
92 {
93     int i;
94     int last_customer;
95     static struct bar_order closing_orders[BAR_STAFF];
96
97     lock_acquire(order_lock);
98     customers_left--;
99     last_customer = customers_left == 0;

```

```

100     lock_release(order_lock);
101
102     if (last_customer) {
103         for (i = 0; i < BAR_STAFF; i++) {
104             closing_orders[i].valid = 0;
105             enqueue_order(&closing_orders[i]);
106         }
107     }
108 }
109
110 struct bar_order *
111 bar_get_order(void)
112 {
113     return dequeue_order();
114 }
115
116 static void
117 sort_bottles(int bottles[], int nbottles)
118 {
119     int i, j, tmp;
120
121     for (i = 0; i < nbottles; i++) {
122         for (j = i + 1; j < nbottles; j++) {
123             if (bottles[j] < bottles[i]) {
124                 tmp = bottles[i];
125                 bottles[i] = bottles[j];
126                 bottles[j] = tmp;
127             }
128         }
129     }
130 }
131
132 void
133 bar_fill_order(struct bar_order *order)
134 {
135     int bottles[DRINK_SIZE];
136     int nbottles;
137     int i;
138
139     nbottles = 0;
140     for (i = 0; i < DRINK_SIZE; i++) {
141         if (order->requested[i] != NO_INGREDIENT) {
142             KASSERT(order->requested[i] >= 1);
143             KASSERT(order->requested[i] <= NBOTTLES);
144             bottles[nbottles++] = order->requested[i];
145         }
146     }
147
148     sort_bottles(bottles, nbottles);
149

```

```

150     for (i = 0; i < nbottles; i++) {
151         if (i == 0 || bottles[i] != bottles[i - 1]) {
152             lock_acquire(bottle_locks[bottles[i]]);
153         }
154     }
155
156     for (i = 0; i < DRINK_SIZE; i++) {
157         order->contents[i] = order->requested[i];
158     }
159
160     for (i = nbottles - 1; i >= 0; i--) {
161         if (i == 0 || bottles[i] != bottles[i - 1]) {
162             lock_release(bottle_locks[bottles[i]]);
163         }
164     }
165
166     V(order->finished);
167 }

```

D Source Code: bar_driver.c

```
1  /* Driver for the ASST1 bar synchronization problem. */
2  #include <types.h>
3  #include <lib.h>
4  #include <synch.h>
5  #include <thread.h>
6  #include <test.h>
7  #include "bar_driver.h"
8
9  static struct semaphore *bar_finished;
10 static int bottle_count[NBOTTLES + 1];
11 static struct lock *stats_lock;
12
13 static void
14 record_bottle_use(const int requested[DRINK_SIZE])
15 {
16     int i;
17
18     lock_acquire(stats_lock);
19     for (i = 0; i < DRINK_SIZE; i++) {
20         if (requested[i] != NO_INGREDIENT) {
21             KASSERT(requested[i] >= 1);
22             KASSERT(requested[i] <= NBOTTLES);
23             bottle_count[requested[i]]++;
24         }
25     }
26     lock_release(stats_lock);
27 }
28
29 static void
30 customer(void *unused, unsigned long customer_id)
31 {
32     int i, j;
33     struct bar_order order;
34     char semname[32];
35
36     (void)unused;
37
38     snprintf(semname, sizeof(semname), "customer %lu",
39              customer_id);
40     order.finished = sem_create(semname, 0);
41     KASSERT(order.finished != NULL);
42
43     for (i = 0; i < DRINKS_PER_CUSTOMER; i++) {
44         order.valid = 1;
45         order.customer_id = customer_id;
46         for (j = 0; j < DRINK_SIZE; j++) {
47             order.requested[j] = NO_INGREDIENT;
48             order.contents[j] = NO_INGREDIENT;
49         }
50     }
51 }
```

```

49         order.requested[0] = BEER;
50         bar_customer_order(&order);
51
52         for (j = 0; j < DRINK_SIZE; j++) {
53             if (order.contents[j] != order.requested[
54                 j]) {
55                 panic("customer %lu received
56                     wrong drink\n",
57                     customer_id);
58             }
59             record_bottle_use(order.requested);
60         }
61
62         sem_destroy(order.finished);
63         bar_customer_done();
64         V(bar_finished);
65     }
66
67     static void
68     staff(void *unused, unsigned long staff_id)
69     {
70         struct bar_order *order;
71         int mixed;
72
73         (void)unused;
74         mixed = 0;
75
76         while (1) {
77             order = bar_get_order();
78             if (!order->valid) {
79                 break;
80             }
81             bar_fill_order(order);
82             mixed++;
83         }
84
85         kprintf("S %lu going home after mixing %d drinks\n",
86                 staff_id, mixed);
87         V(bar_finished);
88     }
89
90     int
91     runbar(int nargs, char **args)
92     {
93         int i, error;
94
95         (void)nargs;
96         (void)args;

```



```

97     bar_finished = sem_create("bar finished", 0);
98     KASSERT(bar_finished != NULL);
99
100     stats_lock = lock_create("bar stats lock");
101     KASSERT(stats_lock != NULL);
102
103     for (i = 1; i <= NBOTTLES; i++) {
104         bottle_count[i] = 0;
105     }
106
107     bar_open();
108
109     for (i = 0; i < BAR_STAFF; i++) {
110         error = thread_fork("bar staff", NULL, staff,
111             NULL, i);
112         if (error) {
113             panic("bar staff: thread_fork failed: %s\
114                 n",
115                 strerror(error));
116         }
117     }
118
119     for (i = 0; i < BAR_CUSTOMERS; i++) {
120         error = thread_fork("bar customer", NULL,
121             customer, NULL, i);
122         if (error) {
123             panic("bar customer: thread_fork failed:
124                 %s\n",
125                 strerror(error));
126         }
127     }
128
129     for (i = 0; i < BAR_STAFF + BAR_CUSTOMERS; i++) {
130         P(bar_finished);
131     }
132
133     for (i = 1; i <= NBOTTLES; i++) {
134         kprintf("Bottle %d used for %d doses\n", i,
135             bottle_count[i]);
136     }
137
138     bar_close();
139     lock_destroy(stats_lock);
140     sem_destroy(bar_finished);
141
142     return 0;
143 }

```

E Source Code: synch.c

```
1  /*
2  * Copyright (c) 2000, 2001, 2002, 2003, 2004, 2005, 2008, 2009
3  *     The President and Fellows of Harvard College.
4  *
5  * Redistribution and use in source and binary forms, with or
6  * without
7  * modification, are permitted provided that the following
8  * conditions
9  * are met:
10 * 1. Redistributions of source code must retain the above
11 *    copyright
12 *    notice, this list of conditions and the following
13 *    disclaimer.
14 * 2. Redistributions in binary form must reproduce the above
15 *    copyright
16 *    notice, this list of conditions and the following
17 *    disclaimer in the
18 *    documentation and/or other materials provided with the
19 *    distribution.
20 * 3. Neither the name of the University nor the names of its
21 *    contributors
22 *    may be used to endorse or promote products derived from
23 *    this software
24 *    without specific prior written permission.
25 *
26 * THIS SOFTWARE IS PROVIDED BY THE UNIVERSITY AND CONTRIBUTORS
27 *    'AS IS' AND
28 *    ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED
29 *    TO, THE
30 *    IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A
31 *    PARTICULAR PURPOSE
32 *    ARE DISCLAIMED. IN NO EVENT SHALL THE UNIVERSITY OR
33 *    CONTRIBUTORS BE LIABLE
34 *    FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
35 *    CONSEQUENTIAL
36 *    DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF
37 *    SUBSTITUTE GOODS
38 *    OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS
39 *    INTERRUPTION)
40 *    HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN
41 *    CONTRACT, STRICT
42 *    LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING
43 *    IN ANY WAY
44 *    OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE
45 *    POSSIBILITY OF
46 *    SUCH DAMAGE.
47 */
48 /*
```

```

31  * Synchronization primitives.
32  * The specifications of the functions are in synch.h.
33  */
34
35  #include <types.h>
36  #include <lib.h>
37  #include <spinlock.h>
38  #include <wchan.h>
39  #include <thread.h>
40  #include <current.h>
41  #include <synch.h>
42
43  //////////////////////////////////////
44  //
45  // Semaphore.
46
47  struct semaphore *
48  sem_create(const char *name, int initial_count)
49  {
50      struct semaphore *sem;
51
52      KASSERT(initial_count >= 0);
53
54      sem = kmalloc(sizeof(struct semaphore));
55      if (sem == NULL) {
56          return NULL;
57      }
58
59      sem->sem_name = kstrdup(name);
60      if (sem->sem_name == NULL) {
61          kfree(sem);
62          return NULL;
63      }
64
65      sem->sem_wchan = wchan_create(sem->sem_name);
66      if (sem->sem_wchan == NULL) {
67          kfree(sem->sem_name);
68          kfree(sem);
69          return NULL;
70      }
71
72      spinlock_init(&sem->sem_lock);
73      sem->sem_count = initial_count;
74
75      return sem;
76  }
77
78  void
79  sem_destroy(struct semaphore *sem)
80  {
81      KASSERT(sem != NULL);

```

```

82
83     /* wchan_cleanup will assert if anyone's waiting on it */
84     spinlock_cleanup(&sem->sem_lock);
85     wchan_destroy(sem->sem_wchan);
86     kfree(sem->sem_name);
87     kfree(sem);
88 }
89
90 void
91 P(struct semaphore *sem)
92 {
93     KASSERT(sem != NULL);
94
95     /*
96      * May not block in an interrupt handler.
97      *
98      * For robustness, always check, even if we can actually
99      * complete the P without blocking.
100     */
101     KASSERT(curthread->t_in_interrupt == false);
102
103     spinlock_acquire(&sem->sem_lock);
104     while (sem->sem_count == 0) {
105         /*
106          * Bridge to the wchan lock, so if someone else
107          * comes
108          * along in V right this instant the wakeup can't
109          * go
110          * through on the wchan until we've finished
111          * going to
112          * sleep. Note that wchan_sleep unlocks the wchan
113          * .
114          *
115          * Note that we don't maintain strict FIFO
116          * ordering of
117          * threads going through the semaphore; that is,
118          * we
119          * might "get" it on the first try even if other
120          * threads are waiting. Apparently according to
121          * some
122          * textbooks semaphores must for some reason have
123          * strict ordering. Too bad. :-))
124          *
125          * Exercise: how would you implement strict FIFO
126          * ordering?
127         */
128         wchan_lock(sem->sem_wchan);
129         spinlock_release(&sem->sem_lock);
130         wchan_sleep(sem->sem_wchan);
131
132         spinlock_acquire(&sem->sem_lock);

```

```

126     }
127     KASSERT(sem->sem_count > 0);
128     sem->sem_count--;
129     spinlock_release(&sem->sem_lock);
130 }
131
132 void
133 V(struct semaphore *sem)
134 {
135     KASSERT(sem != NULL);
136
137     spinlock_acquire(&sem->sem_lock);
138
139     sem->sem_count++;
140     KASSERT(sem->sem_count > 0);
141     wchan_wakeone(sem->sem_wchan);
142
143     spinlock_release(&sem->sem_lock);
144 }
145
146 ////////////////////////////////////////////////////
147 //
148 // Lock.
149
150 struct lock *
151 lock_create(const char *name)
152 {
153     struct lock *lock;
154
155     lock = kmalloc(sizeof(struct lock));
156     if (lock == NULL) {
157         return NULL;
158     }
159
160     lock->lk_name = kstrdup(name);
161     if (lock->lk_name == NULL) {
162         kfree(lock);
163         return NULL;
164     }
165
166     lock->lk_wchan = wchan_create(lock->lk_name);
167     if (lock->lk_wchan == NULL) {
168         kfree(lock->lk_name);
169         kfree(lock);
170         return NULL;
171     }
172
173     spinlock_init(&lock->lk_lock);
174     lock->lk_held = false;
175     lock->lk_holder = NULL;
176

```

```

177         return lock;
178     }
179
180     void
181     lock_destroy(struct lock *lock)
182     {
183         KASSERT(lock != NULL);
184         KASSERT(lock->lk_held == false);
185
186         spinlock_cleanup(&lock->lk_lock);
187         wchan_destroy(lock->lk_wchan);
188         kfree(lock->lk_name);
189         kfree(lock);
190     }
191
192     void
193     lock_acquire(struct lock *lock)
194     {
195         KASSERT(lock != NULL);
196         KASSERT(curthread->t_in_interrupt == false);
197         KASSERT(lock_do_i_hold(lock) == false);
198
199         spinlock_acquire(&lock->lk_lock);
200         while (lock->lk_held) {
201             wchan_lock(lock->lk_wchan);
202             spinlock_release(&lock->lk_lock);
203             wchan_sleep(lock->lk_wchan);
204             spinlock_acquire(&lock->lk_lock);
205         }
206         lock->lk_held = true;
207         lock->lk_holder = curthread;
208         spinlock_release(&lock->lk_lock);
209     }
210
211     void
212     lock_release(struct lock *lock)
213     {
214         KASSERT(lock != NULL);
215         KASSERT(lock_do_i_hold(lock));
216
217         spinlock_acquire(&lock->lk_lock);
218         lock->lk_held = false;
219         lock->lk_holder = NULL;
220         wchan_wakeone(lock->lk_wchan);
221         spinlock_release(&lock->lk_lock);
222     }
223
224     bool
225     lock_do_i_hold(struct lock *lock)
226     {
227         bool held;

```

```

228
229     KASSERT(lock != NULL);
230
231     spinlock_acquire(&lock->lk_lock);
232     held = lock->lk_held && lock->lk_holder == curthread;
233     spinlock_release(&lock->lk_lock);
234
235     return held;
236 }
237
238 ///////////////////////////////////////////////////
239 //
240 // CV
241
242
243 struct cv *
244 cv_create(const char *name)
245 {
246     struct cv *cv;
247
248     cv = kmalloc(sizeof(struct cv));
249     if (cv == NULL) {
250         return NULL;
251     }
252
253     cv->cv_name = kstrdup(name);
254     if (cv->cv_name == NULL) {
255         kfree(cv);
256         return NULL;
257     }
258
259     cv->cv_wchan = wchan_create(cv->cv_name);
260     if (cv->cv_wchan == NULL) {
261         kfree(cv->cv_name);
262         kfree(cv);
263         return NULL;
264     }
265
266     return cv;
267 }
268
269 void
270 cv_destroy(struct cv *cv)
271 {
272     KASSERT(cv != NULL);
273
274     wchan_destroy(cv->cv_wchan);
275     kfree(cv->cv_name);
276     kfree(cv);
277 }
278

```

```

279 void
280 cv_wait(struct cv *cv, struct lock *lock)
281 {
282     KASSERT(cv != NULL);
283     KASSERT(lock != NULL);
284     KASSERT(lock_do_i_hold(lock));
285     KASSERT(curthread->t_in_interrupt == false);
286
287     wchan_lock(cv->cv_wchan);
288     lock_release(lock);
289     wchan_sleep(cv->cv_wchan);
290     lock_acquire(lock);
291 }
292
293 void
294 cv_signal(struct cv *cv, struct lock *lock)
295 {
296     KASSERT(cv != NULL);
297     KASSERT(lock != NULL);
298     KASSERT(lock_do_i_hold(lock));
299
300     wchan_wakeone(cv->cv_wchan);
301 }
302
303 void
304 cv_broadcast(struct cv *cv, struct lock *lock)
305 {
306     KASSERT(cv != NULL);
307     KASSERT(lock != NULL);
308     KASSERT(lock_do_i_hold(lock));
309
310     wchan_wakeall(cv->cv_wchan);
311 }

```